Data Mining 1 Project

Giulia Fabiani Camilla Maffeis

A.Y. 2024/2025

CONTENTS

1 DATA UNDERSTANDING AND PREPARATION

The goal of the project is to analyze the IMDb Dataset, which contains data about movies, TV shows, and other forms of visual entertainment, along with their ratings. Each record includes key information about the title, insights into critical aspects such as awards and reviews, as well as statistical ratings and other metadata. The dataset is updated as of September 1, 2024.

1.1 Data Semantics and First Global Exploration of the Dataset

The IMDb Dataset contains 16431 records and 23 attributes, among which 10 are categorical and 13 numeric. They are listed in Tables 3 and 4. Among the **numerical attributes**, most of them are discrete and ratio-scaled, as they represent counts, with *startYear* and *endYear* being interval values. Only *runtimeMinutes* can be classified as a continuous attribute, as it represents a time duration, although it only takes on integer values in the dataset. Among the **categorical attributes**, *canHaveEpisodes*, *isRatable* and *isAdult* are **binary**, *rating*, *worstRating* and *bestRating* are **ordinal**, while the rest are all **nominal**.

Looking at the records' data types, we can observe some syntactic inaccuracies: *isAdult* should be converted from integer to boolean, whereas *rating* assumes ten categorical values of the form *'(0, 1]'*, *'(1, 2]'*, ..., *'(9, 10]'*. We can assume they represent the binning of a continuous rating, therefore we create the variable *ratingNum* to map them onto an integer space [1,10] to facilitate later analysis. *countryOfOrigin* and *genres* contain collections of categorical values; therefore, we transform them into lists for easier handling. Some discrete numerical attributes (*endYear*, *awardWins*) are stored as float because `int64` does not support NaN values; however, we do not deem it necessary to convert them.

A first global analysis of the dataframe's summary and descriptive statistics already gives us valuable insight into which variables have missing values[1], and into which features are uninformative: *bestRating*, *worstRating*, and *isRatable* can be safely dropped, as they only take one value for all records (10, 1, and 1, respectively). We can also see that *numVotes* and *ratingCount* share very similar, when not identical, statistics, so we drop *ratingCount* and only keep *numVotes*.

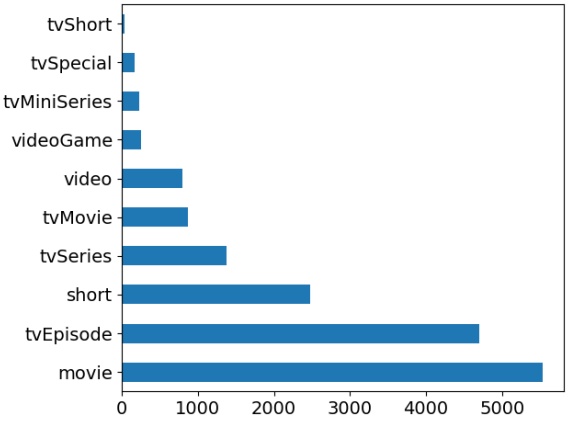


Figure 1: Barplot for *titleType*.

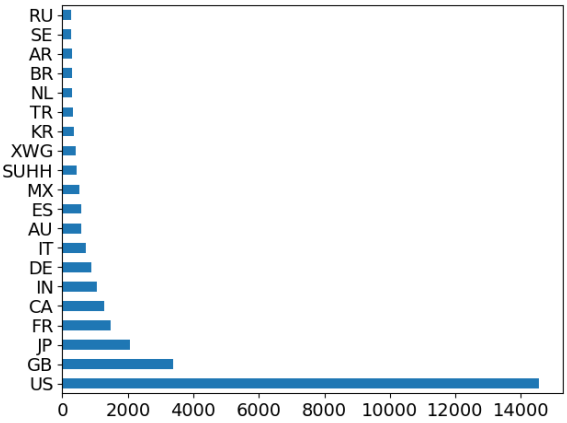


Figure 2: Barplot for *countryOfOrigin* (top 20).

1.2 Distribution of Categorical Variables and Statistics

In an effort to analyze the univariate distribution of categorical variables, we produced barplots for *titleType* (Fig. 1), *countryOfOrigin* (Fig. 2), and *genres* (Fig. 3). All distributions are imbalanced, being dominated by few very common classes: *movie* and *tvepisode*, the *US*, *drama* and *comedy*, respectively. A barplot for *rating*[2], in Fig. 4, shows a left-skewed distribution, with the mode being the [7,8) category.

By looking at the categories for the different variables, we question the informativeness of *canHaveEpisodes* and *isAdult*. The first takes on positive values only for the title types *tvSeries* and *tvMiniSeries*: we keep it, as an explicit feature of

1 As the missing values were recorded inconsistently in the dataset, we used the `read_csv` function, passing a list of strings to recognize as missing values to the parameter `na_values`.

2 In this data exploration phase, we analyzed this ordinal variable both from a categorical and a numeric point of view, as to maximize insights.

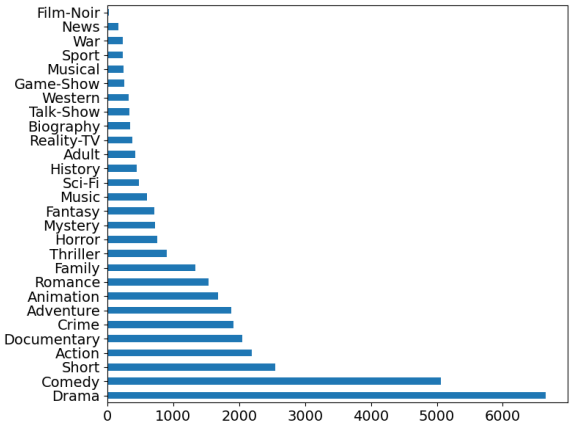


Figure 3: Barplot for *genres*.

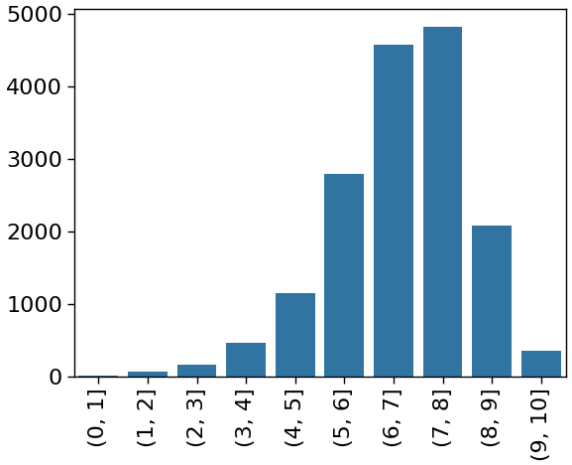


Figure 4: Barplot for *rating*.

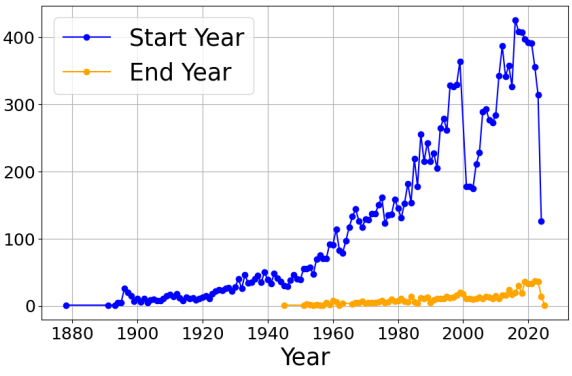


Figure 5: Time series: number of titles being released and ended by year.

serialized content. *isAdult* proves to be redundant as its information is conveyed by the *adult* category in *genres*, therefore we drop it.

1.3 Distribution of Numeric Variables and Statistics; Transformation of Variables

Before analyzing numeric variables, we add the following features:

- *numCountriesOfOrigin*: the number of countries listed for the record for the variable *countryOfOrigin*. For the training set, the values range from 1 to 10.
- *numGenres*: the number of genres listed in *genres*. For the training set, the values range from 1 to 3.
- *reviewsTotal*: the sum of *criticReviewsTotal* and *userReviewsTotal*.
- *criticReviewsRatio*: the proportion of review from critics (*criticsReviewTotal*) on the total number of reviews of a record (*reviewsTotal*). For unreviewed records, it is set to zero.
- *awardsAndNominations*: the sum of *awardWins* and *awardNominationsExcludeWins*.

Moreover, we use the available interval features, *startYear* and *endYear* to generate the timeline graph in Fig. 5. We can observe how the number of titles being released each year has consistently grown until the end of the 2010s, with two noticeable dips in this trend: around 2001 (which could be linked to the aftermath of 9/11) and after 2020 (probably due to the Covid-19 pandemic). The time series for *endYear* follows the same trend on a smaller scale: we suspect a positive correlation between the two variables. There are no end dates available before the 1940s, which is understandable, as serialized content was not produced until around the 1930s.

In order to have a first global view of both the univariate distribution of the numeric variables singularly, and of possible interesting pairwise correlations, we build a pairplot with a KDE graph for each variable in the diagonal. Our previous intuition about a positive correlation between *startYear* and *endYear* is proved by the scatterplot in Fig. 6, therefore we drop *endYear*, which is more problematic due to the large number of missing values.

The outliers visible in this plot highlight supposedly particularly long-running television shows: as a matter of fact, of those we analyzed, only the

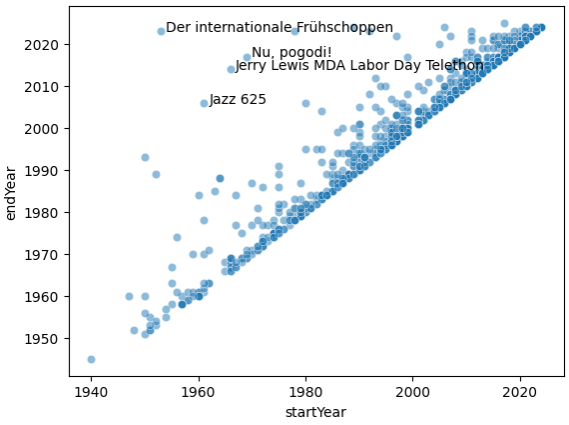


Figure 6: Scatterplot endYear~startYear.

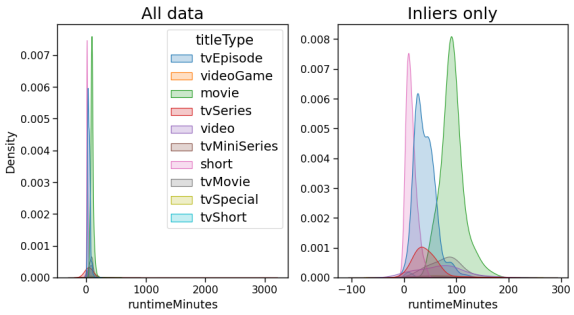


Figure 7: KDE for runtimeMinutes per title type, including and excluding outliers.

MDA Labor Day Telethon run continuously for 45 years, while the other titles refer to shows that were renewed or re-broadcasted after a long hiatus.

Most numeric features (*awardWins*, *numVotes*, *totalImages*, *totalVideos*, *totalCredits*, *criticReviewsTotal*, *awardNominationsExcludeWins*, *awardsAndNominations*, *numRegions*, *userReviewsTotal*, *numCountriesOfOrigin*, *reviewsTotal*) have heavily right-skewed distributions; it makes sense that their distribution would approximate a power law. We can address this problem by using a log transformation of these features.

On the other hand, *runtimeMinutes* also has a right-skewed distribution, but when excluding outliers (values higher than 3rd quartile + 1.5 IQR), we can see that, for the most part, runtimes are dependent on the *titleType* and that the distribution of the runtime for each title type approximates a bell curve (Fig. 7); therefore, we prefer not to log transform this variable and be mindful of the presence of outliers. Examining the titles with a longer runtime reveals that part of the outliers is due to the inconsistent strategy used for the computation of the runtime for serialized titles (series and miniseries): in some cases, the episode runtime is provided, in others, it reports the runtime for the entire series, i.e. the sum of all its episodes' runtimes.

1.4 Handling Missing Values

Four variables in the dataset have been found to contain missing values: *endYear* (15,617), *runtimeMinutes* (4,852), *awardWins* (2,618), *genres* (382).

In Subsection 1.3, we opted to drop *endYear* because its value was missing for most records. This decision is corroborated by the fact that, on one side, it heavily correlates with *startYear*, and on the other, the only records that do have an end year are of the type *tvSeries* and *tvMiniSeries*, and the information about the serial nature of media is already carried by the feature *canHaveEpisodes*.

awardWins has almost three thousand missing values. Given its highly unbalanced distribution, with the large majority of the records having no awards, it makes sense to fill them with the most frequent value, i.e. zero.

Analyzing the distribution of *genres*, we noticed that there is some variation in the frequency of different genres w.r.t. *titleType*: for example, while *Drama* is overall the most frequent genre (as can be seen in Fig. 3), *Comedy* tends to be more prevalent in serialized content. We decided to fill the 382 missing values for *genres* with the most frequent list of genres given the title type of the record (cf. Table 1).

As previously mentioned in Subsection 1.3, *runtimeMinutes* is also dependent on the *titleType* of a record (cf. Fig. 7): movies are generally longer than TV episodes, and shorts are typically shorter than both. We can thefore use the median runtime for the respective *titleType* to fill *runtimeMinutes*' missing values (cf. Table 1).

titleType	top genre	mdn runtime
movie	[Drama]	90
short	[Short]	12
tvEpisode	[Comedy]	40
tvMiniSeries	[Drama]	60
tvMovie	[Drama]	86
tvSeries	[Comedy]	31
tvShort	[Animation, Family, Short]	10
tvSpecial	[Comedy]	60
video	[Adult]	76
videoGame	[Action, Adventure, Fantasy]	28

Table 1: Top genre and median runtime (in minutes) for each *titleType*.

Value	awAndNoms	hasVids	moreCounOfOr
False	13692	14821	15285
True	2739	1610	1146

Table 2: Boolean value counts for *awardsAndNominations*, *hasVideos* and *moreCountriesOfOrigin*.

1.5 Pairwise Correlations and Elimination of Variables

We observed pairwise linear correlation among numerical variables (after operating the log transformation detailed in Subsection 1.3) by computing their correlation heatmap, displaying Pearson's correlation coefficient for each pair of features (Fig. 8, left).

numVotes, *userReviewsTotal*, *criticReviewsTotal* and *reviewsTotal* are all highly positively correlated with each other (with a correlation coefficient greater than 70): we only keep *numVotes*, which has the least skewed distribution.

The majority of records lack awards or nominations, do not have videos, and list only one country of origin. On that account, we binarized *awardsAndNominations*, dropping both *awardWins* and *awardNominationsExcludeWins*. Similarly, we also dropped *totalVideos* and *numCountriesOfOrigin*, replacing them with the boolean features *hasVideos* and *moreCountriesOfOrigin*, respectively (cf. Table 2).

The total number of final features was reduced from 23 to 18; they are listed in Tables 5 and 6. Among the remaining numeric features, the heatmap in Fig. 8 (right) shows that *numVotes* exhibits a moderate to high positive correlation with *totalImages* (0.61), a moderate correlation with *numRegions* (0.56), and a low to moderate correlation with *totalCredits* (0.51).

2 CLUSTERING

We analyzed the dataset using centroid-based, density-based and hierarchical clustering algorithms. For all experiments, we used all numeric features (cf. Table 5), log-transformed if necessary as detailed in Subsec. 1.3, normalized using StandardScaler.

2.1 Centroid-based Clustering

In analyzing the dataset using centroid-based methods, we experimented with two different algorithms, K-Means and Bisecting K-Means. For each experiment, we followed the same steps:

1. Selecting the best value for k: we used both the Elbow method and the Silhouette method as heuristics to determine the appropriate number of clusters. To do so, we iteratively ran the algorithm for k in the range [2,50], plotting the SSE and Silhouette score for each k and comparing the two curves to find the best trade-off.

2. Fitting the algorithm and evaluating the resulting clustering in terms of cohesion and separation by calculating the SSE, Silhouette score, and correlation between the distance matrix of the dataset and the ideal similarity matrix.

3. Extrinsic evaluation: analysis of cluster centroids and of the clusters' composition w.r.t. categorical and numeric features.

2.1.1 *K-Means*

We selected **k=7** as a trade-off between SSE and Silhouette score. The resulting **SSE** and **Silhouette score** are **62588.082** and **0.189**, respectively, while the **correlation** is **-0.458**. Though the SSE value might seem high, it is significantly lower than the SSE of 500 randomly generated datasets (cf. Fig. 9), which suggests that our clustering is meaningful and not a product of chance.

The clusters' composition is reported in Table 7: in terms of population, we can distinguish Cluster 0 as the largest cluster, with almost 5000 records, followed by Cluster 2 with over 3000 records. The smallest cluster is Cluster 6, with only 950 records. The remaining clusters have between 1400 and 2000 records.

The first thing that emerges from the parallel coordinates graph (Fig. 10) is how Cluster 3 is well-separated from the others w.r.t half of the features: it has a particularly high value for *numVotes*, *totalImages*, *totalCredits*, and *numRegions* — variables for which we had observed moderate correlation in Subsec. 1.5. Cluster 5 has a significantly higher value for *criticReviewsRatio*: the algorithm might have isolated titles with greater critical appeal. Clusters 6 and 4 have the lowest runtime by far (median: 12 minutes); Cluster 6, the smallest one, is also characterized by a low value for *startYear* and *totalCredits*: it probably contains the oldest and shortest titles. Clusters 0 and 2 appear to be the least-well separated from the others; they are indeed the biggest clusters.

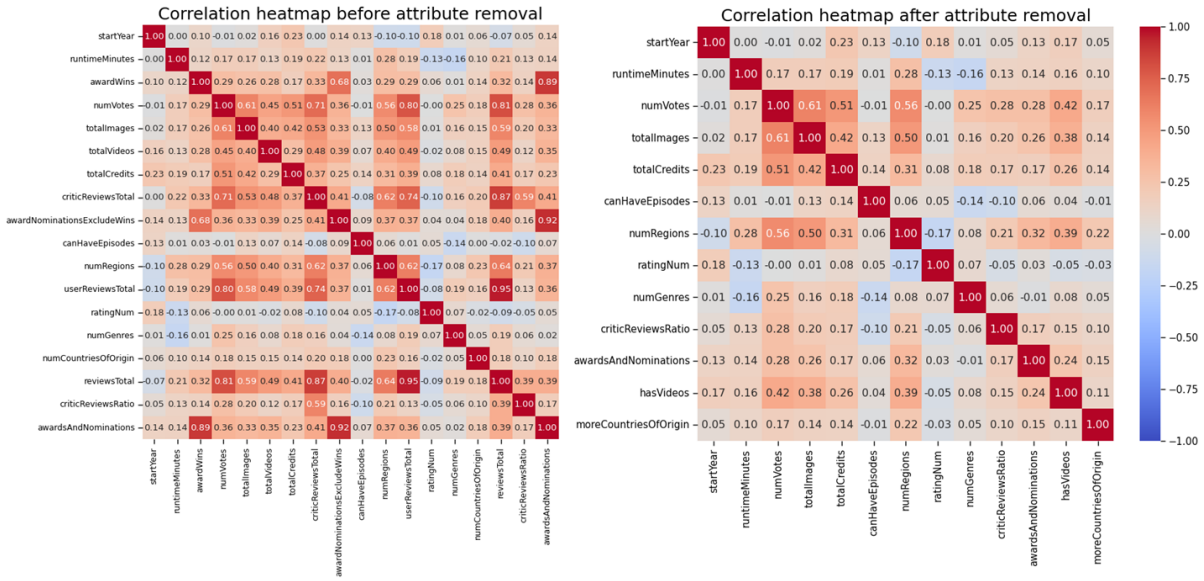


Figure 8: Correlation heatmaps, before and after attribute removal.

Feature	Description	Type	dtype
runtimeMinutes	Primary runtime of the title, in minutes.	Continuous. Ratio-scaled.	float64
startYear	Represents the release year of a title. In the case of TV Series, it is the series' start year.	Discrete. Interval.	int64
endYear	TV Series end year.	Discrete. Interval.	float64
ratingCount	The total number of user ratings submitted for the title.	Discrete. Ratio-scaled.	int64
numVotes	Number of votes the title has received.	Discrete. Ratio-scaled.	int64
numRegions	The regions number for this version of the title.	Discrete. Ratio-scaled.	int64
totalImages	Total Number of Images for the title within the IMDb title page.	Discrete. Ratio-scaled.	int64
totalVideos	Total Number of Videos for the title within the IMDb title page.	Discrete. Ratio-scaled.	int64
totalCredits	Total Number of Credits for the title.	Discrete. Ratio-scaled.	int64
criticsReviewTotal	Total Number of Critic Reviews.	Discrete. Ratio-scaled.	int64
awardWins	Number of awards the title won.	Discrete. Ratio-scaled.	float64
awardNominationExclWins	Number of award nominations excluding wins.	Discrete. Ratio-scaled.	int64
userReviewsTotal	Total Number of Users Reviews.	Discrete. Ratio-scaled.	int64

Table 3: Numeric attributes.

Feature	Description	Type	dtype
originalTitle	Original title, in the original language.	Categorical; mostly unique classes (i.e., names).	object
titleType	The type/format of the title (e.g., movie, short, episode, video, etc.).	Categorical, 10 classes.	object
countryOfOrigin	The country where the title was primarily produced. Some titles can belong to more than one class.	Categorical, 153 classes. A record can belong to more than one class.	object
genres	The genre(s) associated with the title (e.g., drama, comedy, action). Some titles can belong to more than one class.	Categorical, 29 classes. A record can belong to more than one class.	object
canHaveEpisodes	Whether or not the title can have episodes.	Asymmetric binary.	bool
isRatable	Whether or not the title can be rated by users.	Asymmetric binary.	bool
isAdult	Whether or not the title is for adults.	Asymmetric binary.	int64
rating	IMDB title rating class.	Ordinal, 10 values.	object
worstRating	Worst title rating.	Ordinal, supposedly [1,10].	int64
bestRating	Best title rating.	Ordinal, supposedly [1,10].	int64

Table 4: Categorical, ordinal, and binary attributes.

Feature	Description	Type
runtimeMinutes	Primary runtime of the title, in minutes.	Continuous. Ratio-scaled.
startYear	Represents the release year of a title. In the case of TV Series, it is the series' start year.	Discrete. Interval.
numVotes	Number of votes the title has received.	Discrete. Ratio-scaled. Log-transformed.
numRegions	The regions number for this version of the title.	Discrete. Ratio-scaled. Log-transformed.
totalImages	Total number of images for the title within the IMDb title page.	Discrete. Ratio-scaled. Log-transformed.
totalCredits	Total number of credits for the title.	Discrete. Ratio-scaled. Log-transformed.
criticReviewsRatio	The proportion of reviews from critics on the total number of reviews of a record. For unreviewed records, it is set to 0.	Continuous. Ratio-scaled.
numGenres	The number of genres listed for the variable *genres*.	Discrete. Ratio-scaled.

Table 5: Numeric attributes after data preparation.

Feature	Description	Type
originalTitle	Original title, in the original language.	Categorical; mostly unique classes (i.e. names).
titleType	The type/format of the title (e.g. movie, short, tvseries, tvepisode, video, etc.).	Categorical, 10 classes.
countryOfOrigin	The country where the title was primarily produced.	Categorical, 153 classes. Some titles can belong to more than 1 class.
genres	The genre(s) associated with the title (e.g., drama, comedy, action).	Categorical, 29 classes. Some titles can belong to more than 1 class.
rating	IMDB title rating class.	Ordinal, 10 values.
ratingNum	Mapping of *rating* to the integers [1,10].	Ordinal, 10 values.
awardsAndNominations	Whether the title was at least nominated for an award.	Asymmetric binary.
canHaveEpisodes	Whether the title can have episodes.	Asymmetric binary.
hasVideos	Whether the title IMDb title page has at least one video.	Asymmetric binary.
moreCountriesOfOrigin	Whether the title was produced in more than one country.	Binary.

Table 6: Categorical, ordinal, and binary attributes after data preparation.

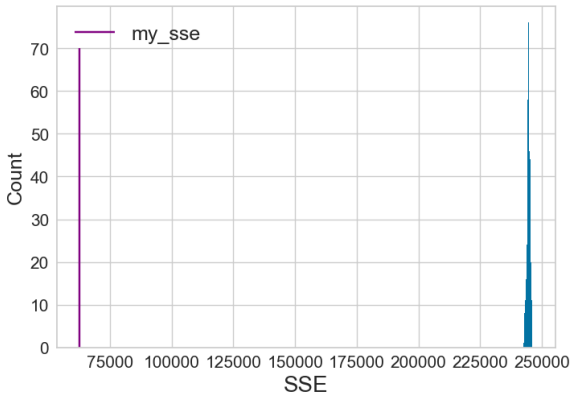


Figure 9: Statistical framework for clusters' validity: SSE (K-Means).

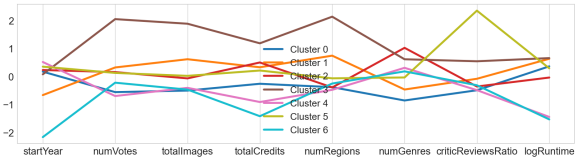


Figure 10: Centroids' visualization for K-Means.

Clust.	# of records
0	4841
1	1985
2	3424
3	1424
4	1985
5	1822
6	950

Table 7: Clusters for K-Means.

Clust.	# of records
0	1881
1	1398
2	1397
3	1712
4	3595
5	4184
6	2264

Table 8: Clusters for BK-Means.

In order to evaluate the results, we visualized the distribution of the clusters against known categorical features.

- *titleType* (cf. the heatmap in Fig. 11): the smallest cluster, Cluster 6, is the purest, being mainly made up of shorts. Nevertheless, it's Cluster 4 that contains the largest number of shorts (as well as some tvEpisodes). It also stands out for having the fewest movies -— a class otherwise well-represented across all other clusters. Cluster 2 has a very large proportion of tvEpisode; indeed its median runtime is 40 minutes. The importance of *logRuntime* for explaining this clustering is also highlighted by the fact that the few tvShorts in the training set have been almost entirely dis-

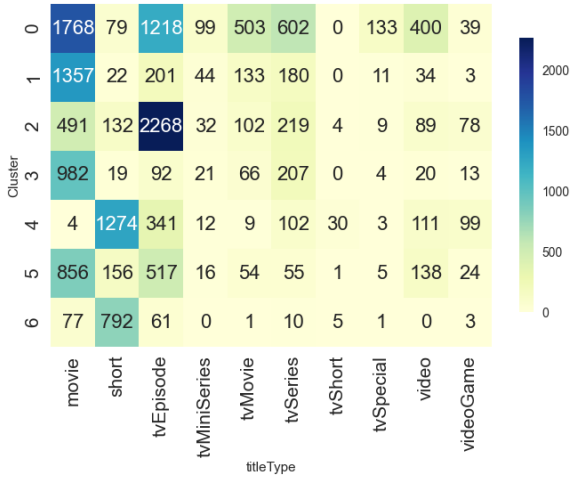


Figure 11: Heatmap Clusters vs titleType for K-Means.

tributed amongst these three clusters. Cluster 0, the biggest, contains a great number of almost all *titleTypes*.

- Boolean variables: Cluster 3 is the only cluster with a significant proportion of True value for *hasVideos* (53.72% of its records) and *awardsAndNominations* (52.74%). Given the insights from the parallel coordinates plot, we can assume this cluster contains mainstream, popular content: it has relatively more videos, images, and votes on its page and was distributed in more countries, but it doesn't have the highest ratio of critical reviews. Cluster 6, on the other hand, has little to no True values for these variables. The highest proportion of True values for *canHaveEpisodes* is in Cluster 0, which contains the largest amount of tvSeries and tvMiniSeries.

We also analyzed the clusters' composition w.r.t. the numeric variables used by the algorithm, expanding the insights gained from the parallel coordinates plot. Interestingly, the two purest clusters w.r.t to *numGenres* are the biggest: Cluster 0 mostly has one genre, with a minority of two genres, while Cluster 2 mostly has three genres, with a small proportion of two genres. The other clusters are more heterogeneous, having all three possible values. The line graph in Fig. 12 shows the distribution of the clusters w.r.t. *startYear*: as hinted by its centroid, Cluster 6 contains the majority of records released before 1930s, and has no record released after 1983. On the other hand, Cluster 4 only has records released since 1963: the algorithm has therefore grouped older shorts in a cluster and newer shorts in another. Cluster 1, which also had a relatively low centroid, mostly contains records from the 1930s until the 1990s.

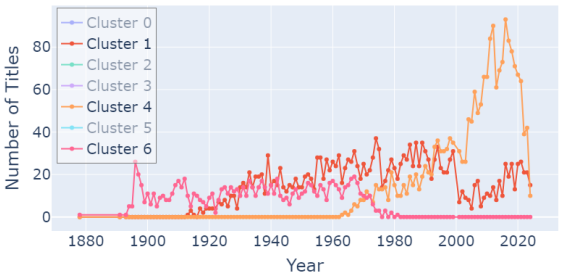


Figure 12: Number of titles by year for relevant clusters for K-Means.

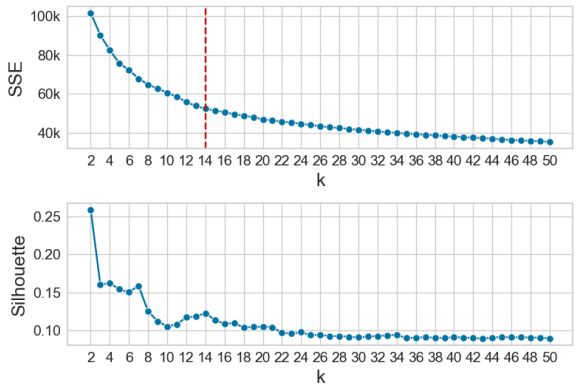


Figure 13: SSE and Silhouette score for selecting k (BK-Means).

2.1.2 *Bisecting K-Means*

We select **k=7**, the only significant local peak in the Silhouette plot (Fig. 13). All evaluation metrics are worse than the ones for K-Means, with a **SSE** equal to **67857.023**, a **Silhouette score** equal to **0.128**, and a **correlation** of **-0.393**. Again, we compared the SSE with the one of 500 randomly generated datasets to confirm its meaningfulness. The clusters' sizes are more homogeneous than K-Means' (cf. Table 8): Cluster 5 is the biggest with around 4000 records, followed by Cluster 4 with almost 3600 records. All other clusters range between around 1400 and 2300 records.

In analyzing the clustering w.r.t. categorical and numeric features, many parallels to K-Means emerged, as well as some differences, especially w.r.t. their *startYear* distribution (cf. Figg. 14, 15 and 16).

We can identify **Cluster 1** and **Cluster 2** as the "higher critical appeal" and "mainstream and popular content" clusters, respectively, similarly to K-Means' Clusters 5 and 3. Noticeably, Cluster 2 is the only cluster with almost no tvEpisodes.

Cluster 3 is the only "shorts cluster", containing around half of all shorts, as well as a small proportion of tvEpisodes and, noticeably, almost no

Figure 14: Centroids' visualization for BK-Means.

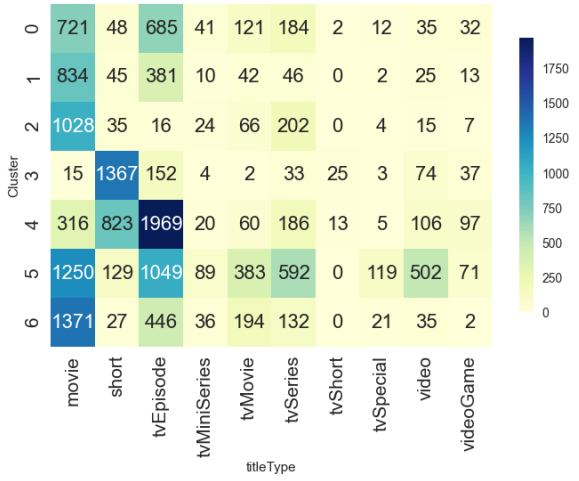


Figure 15: Heatmap Clusters vs titleType (BK-Means).

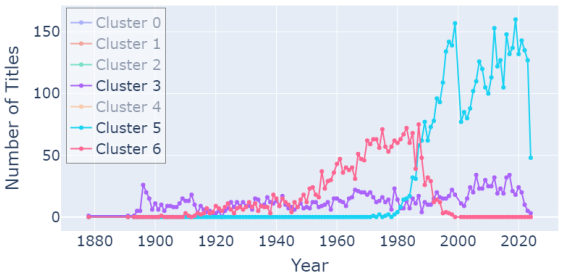


Figure 16: Number of titles by year for relevant clusters for BK-Means.

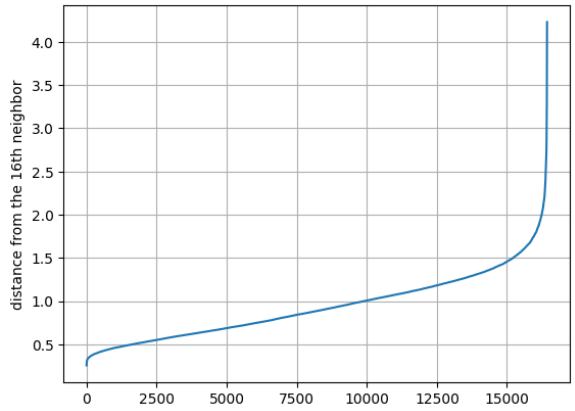


Figure 17: Sorted distances from the 16th neighbor.

movies. Indeed, it characterized by a significantly lower runtime (median: 11 minutes) and *totalCredits* centroid value. This time, the algorithm grouped older and newer short titles together: it does account for the majority of records released before the 1930s (hence having the lowest centroid value for *startYear*), but it contains records from all years.

Again, the three largest clusters are the purest w.r.t. *numGenres*. **Cluster 4** has a relatively low runtime (median: 30 minutes): it's mostly made up of tvEpisodes and of a smaller, yet significant, proportion of shorts; it also contains 37.5% of all videogames. **Cluster 5**, the biggest cluster, only contains newer titles, released since the 1980s. It contains the largest part of the remaining *titleTypes*. **Cluster 6** contains the largest number of movies, despite being only the third-largest cluster overall. It exclusively comprises titles released from the 1910s until 1999, and contains the majority of records released between 1955 and 1987.

In conclusion, although Bisecting K-Means produced worse metrics than K-Means, it still resulted in a meaningful clustering, comparable to that of K-Means. Both methods yielded one or two clusters dominated by shorts, one cluster containing a large portion of tvEpisodes, another capturing mainstream/popular content, and a cluster with critically interesting titles. Neither clustering is separated w.r.t. the *rating*.

2.2 Density-based Clustering

2.2.1 DBSCAN

In order to choose the hyperparameters ϵ and *MinPts*, we experimented with plotted the sorted distances from the k^{th} neighbor for several values of k (powers of 2 between 8 and 512) and checked for the presence of an elbow in the plot. We then fixed *MinPts* to a value of k (we choose 16, since all values of k gave use very similar curves) and used the elbow in the plot to choose a suitable value for ϵ.

In Fig. 17 we can see that the elbow of the curve is around 1.5, so we run the clustering algorithm with *MinPts*=16 and ϵ=1.5. DBSCAN classified almost 99% of our titles as noise points and clustered together all remaining titles.

2.2.2 OPTICS

Since we failed to detect significative clusters in the data determining ϵ with the elbow method, we decided to run OPTICS with the same value of *MinPts* and inspect the reachability plot.

In Fig. 18 we can see that the reachability plot doesn't show any real valleys which would allow us to detect significant clusters. It is possible that

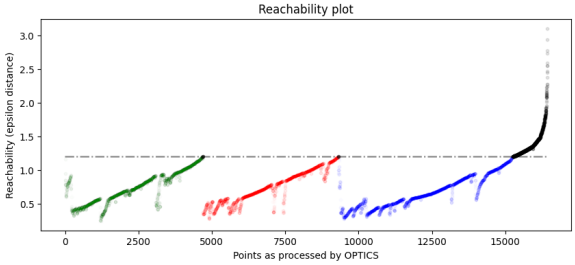


Figure 18: Reachability plot produced by OPTICS with *MinPts*=16

our data is simply too noisy to be suitable for this type of analysis. Still, setting ϵ to 1.2, we were able to extract 3 clusters of similar sizes and remove 1188 noise points. This clustering obtained a Silhouette score of 0.09. The only notable trend found in the analysis of this clustering is that titles are clustered by number of genres (1, 2 or 3).

2.3 Hierarchical Clustering

In analyzing the dataset using agglomerative hierarchical clustering we initially entertained the idea of using a precomputed distance matrix in order to include categorical attributes. However, we ultimately decided against this and used only the numeric features with euclidean distance. This allowed us to obtain silhouette scores comparable with those obtained with centroid based methods, be able to use Ward's method for determining inter-cluster distances and escape the curse of dimensionality. This also enabled us to check if clusters computed on the basis of numerical features were able to infer information conveyed by categorical features.

We performed clustering using all linkage criteria made available from the scikit learn library (single linkage, average linkage, complete linkage and Ward's method), visualized the dendrogram (figg. 19-22) for each clustering and selected an appropriate threshold value for further analysis.

The **single linkage** criterion failed to yield an interesting clustering, as can be seen from the dendrogram at figure 19. Cutting this dendrogram at any height would create several singleton clusters. This is an expected result, since single linkage performs poorly with noisy data.

Group average performed slightly better (fig 20), but produced a very unbalanced clustering: setting the threshold at 4.7 we obtained 5 clusters, with one single cluster (Cluster 5) containing 98% of the records, as can be seen in fig 23. We obtained a Silhouette score of 0.206.

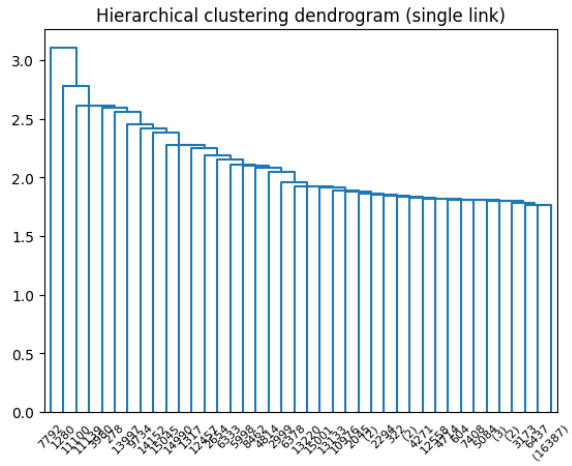


Figure 19: Dendrogram for hierarchical clustering obtained with single linkage.

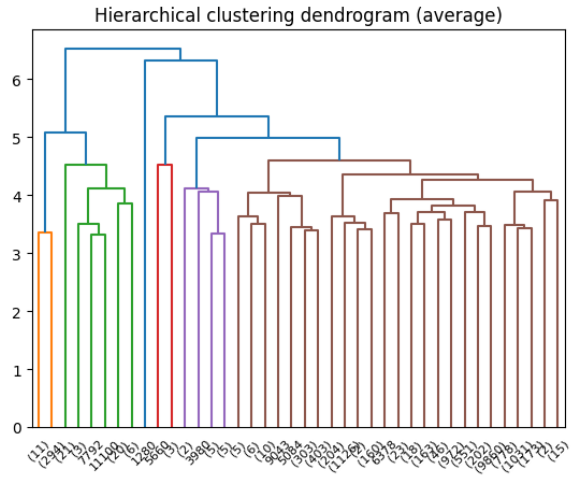


Figure 20: Dendrogram for hierarchical clustering obtained with average linkage (threshold 4.7.)

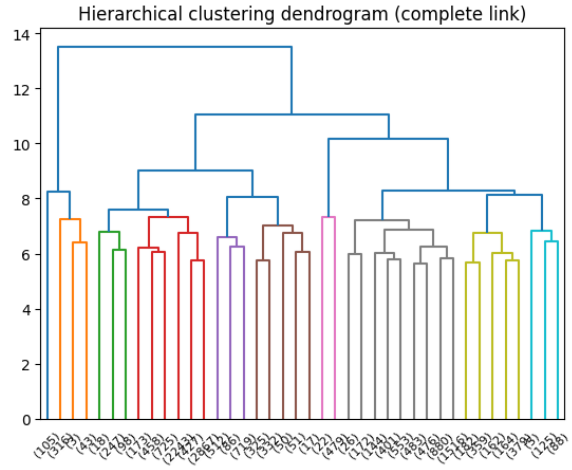


Figure 21: Dendrogram for hierarchical clustering obtained with complete linkage (threshold 7.5).

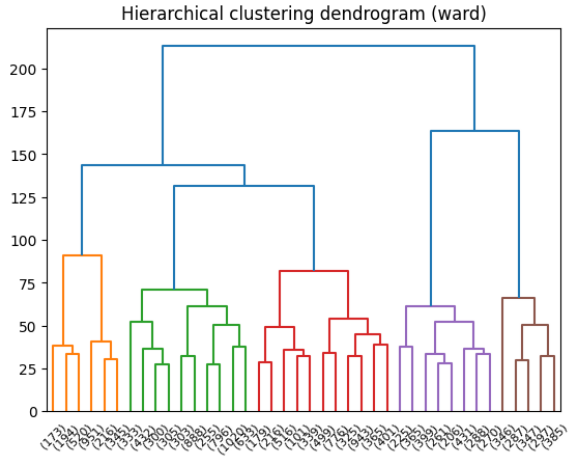


Figure 22: Dendrogram for hierarchical clustering obtained with Ward method (threshold 100).

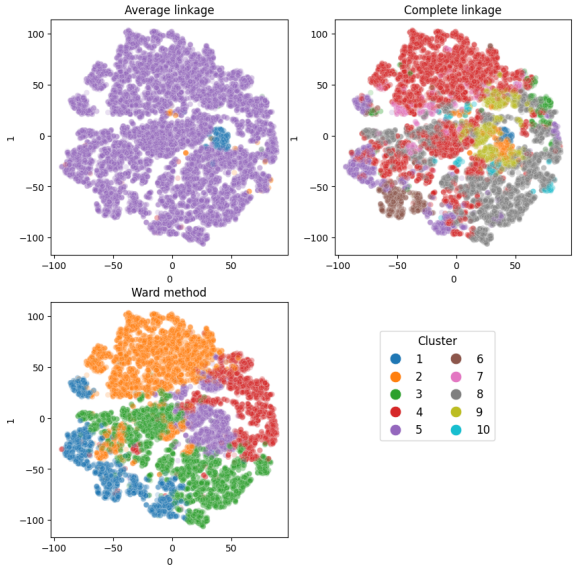


Figure 23: Clusters obtained with different criteria. Visualized by tSNE.

Complete linkage also demonstrated the *rich get richer* tendency of agglomerative clustering, producing clusters of variable size. We cut the dendrogram (Fig. 21) at 7.5, obtaining 10 clusters with two major clusters containing more than 70% of all records and several smaller clusters, six of which having less than 1,000 members. We obtained a Silhouette score of 0.056.

Ward's method obtained the most balanced clustering. We cut the dendrogramm (Fig. 22) at 100, obtaining 5 clusters and a Silhouette score of 0.157.

We then evaluated the clusterings obtained with complete linkage and Ward's method with respect to the distribution of categorical features.

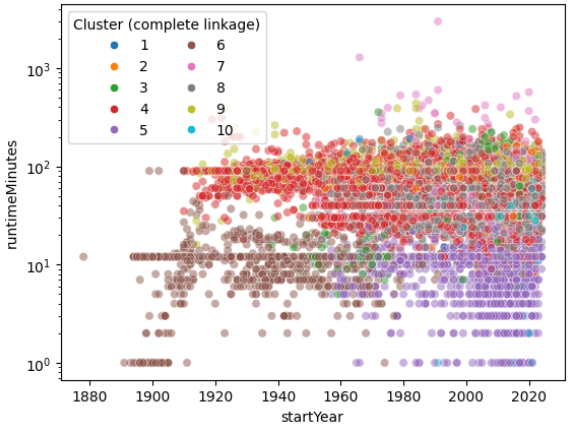


Figure 24: Complete linkage clustering by year of release and runtime.

We found out that both Ward and complete linkage were able to create clusters containing mainly shorts: Ward's Cluster 1 captures 76% of shorts and 85% of TV shorts of the whole dataset and 80% of its members are shorts or TV shorts; for complete linkage, Clusters 5 and 6 capture, in between them, 69% of the total shorts and 87% of TV shorts and are themselves fairly pure with respect to title type (78% of members of Cluster 5 and 94% of members of Cluster 6 are shorts or TV shorts). Fig. 23 shows notable overlap between the shorts clusters obtained by these two methods. We found out that both Ward and complete linkage were able to create some clusters containing mainly shorts: Ward's Cluster 1 captures 76% of shorts and 85% of TV shorts of the whole dataset and 80% of its members are shorts or TV shorts; for complete linkage, Clusters 5 and 6 capture, in between them, 69% of the total shorts and 87% of TV shorts and are themselves fairly pure with respect to title type (78% of members of Cluster 5 and 94% of members of Cluster 6 are shorts or TV shorts). Figure 23 shows notable overlap between the shorts clusters obtained by these two methods.

In fig 24 we show the results of clustering with complete linkage by year of release and runtime. Here we can see that Cluster 4 (which is the most numerous), containing mainly movies and TV episodes is cleanly separated from the two shorts clusters by means of runtime. We can also see that the two clusters of shorts detected by the algorithm differ for date of release: Cluster 6 contains older shorts, while Cluster 5 contains shorts that were released from the 60s onwards. From the dendrogram in Fig. 21, we can see that, had we selected a higher threshold for splitting the

dendrogram, these two clusters would be merged together.

Some of the smallest clusters obtained with complete linkage also proved interesting. In Fig. 25 we can see that Cluster 1 and 2 (that contain mainly movies and some TV series and are merged at a higher threshold) contain titles that are on average more popular that all other titles in the dataset (they have a higher number of votes), but the ones in Cluster 1 are the most popular: this cluster contains blockbusters like *Harry Potter and the Deathly Hallows – Part 2*, *Il buono, il brutto, il cattivo* and *Full metal jacket*, just to name a few, as well as very popular TV series like *The Witcher* and *Euphoria*.

Moreover, almost all (96%) records from cluster 1 received awards or nominations and almost all of them (94%) have attached videos on their IMDB page. Of the slightly less popular records from Cluster 2, 65% received awards or nominations, while 72% have videos on their IMDB page, while the values for the entire database are much lower (respectively 17% and 10%).

Similar results emerge in the clustering with average linkage, where 83% of the 305 records of Cluster 1 (which in figure 23 is shown to overlap significantly with the previous two clusters) received awards and nominations and 91% have attached videos on their IMDB page.

It should be mentioned that the aforementioned clusters capture only a small part of all records that received awards or nominations, but this is, nevertheless, an interesting result, since we only used the numerical features for the clustering, so no information about awards or videos was provided to the model.

2.4 Conclusions

Hierarchical clustering with average linkage obtained the best silhouette score amongst all clustering algorithms we used, but created clusters of very unbalanced sizes, therefore the best clustering results were obtained with **K-means**.

Our data proved unsuitable for analysis with density-based methods, which obtained disappointing results, especially considering their computational cost.

Both centroid-based and hierarchical methods gave us valuable insights into the composition of our dataset, in particular creating some clusters containing mainly shorts and clustering together the most popular titles in the dataset. Moreover, the inspection of the dendrograms produced by hierarchical clustering revealed interesting relationships between clusters.

3 CLASSIFICATION

In this section, we examine the performance of various classification methods w.r.t. two multi-class classification tasks.

Since we noticed promising trends in the cluster analysis, we chose *titleType* as target variable for the first task.

In the second task, we attempted to predict the rating of a title. Since we wanted to avoid having a great number of classes with very low support, we binned the rating values into four categories, each containing approximately the same number of records, as follows:

LOW rating lower than 6

MEDIUM-LOW rating between 6 and 7

MEDIUM-HIGH rating between 7 and 8

HIGH rating 8 or higher

For this second task we expected overall worse results, because none of our features showed high correlation with rating, and we also saw that our records didn't cluster by rating in any significant way.

In evaluating our models, we considered not only the accuracy score, but also the macro average of F1 scores obtained for each class. Since the classes for the first have variable support, we reasoned that this second metric would be a better indicator of the overall performance of the models.

In order to check that the performance of our models was better than random, we created a baseline using a dummy classifier with strategy stratified (which for each test sample predicted a random class with probability equal to the class prior probability). The dummy obtained accuracy .233 and macro F1 .102 on the first task and accuracy .26 and macro F1 .25 on the second task.

Additionally, for each model we plotted the Precision-Recall (PR) curves for each class using the one-vs-rest strategy and reported its PR AUC. We chose PR over ROC because PR curves provide more informative insights for our imbalanced classification problems, in which the number of negative examples significantly outnumbers the positives (especially since we used one-vs-rest).

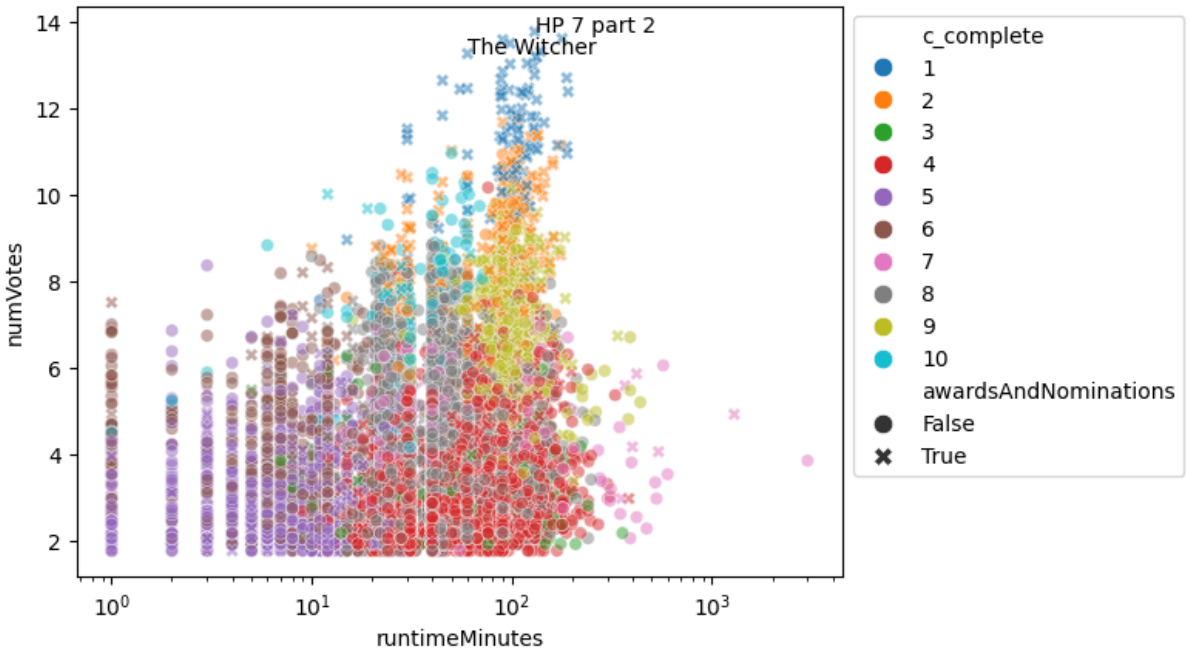


Figure 25: Complete linkage clustering by runtime and number of votes

3.1 K-Nearest Neighbor Classifier

For each target feature we trained the KNN classifier on all numerical features (log transformed if necessary) which were normalized using a standard scaler.

In both cases we performed grid search in order to find the optimal configuration of hyperparameters: we tested different values of k between 2 and 500, two different distance functions (manhattan and euclidean) and whether to weight exemplars by distance. We evaluated the performance of each configuration of hyperparameters performing a 5-fold stratified cross validation on the training set and checking the mean accuracy. In order to limit the time spent on this task, we performed the search in two steps, as illustrated by figg. 26 and 27:

1. We performed grid search with values of k between 2 and 500 using a step of 5;
2. After visually inspecting the trend of the mean accuracy on our first grid search by means of a lineplot, we refined our search for k to a restricted range, this time using a step of 1.

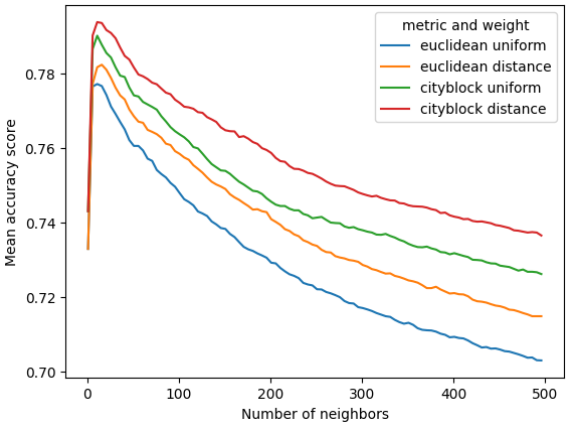


Figure 26: Accuracy of knn models evaluated during grid search (target: titleType).

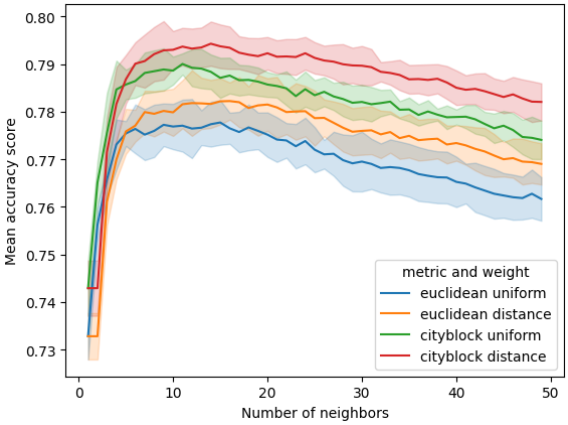


Figure 27: The search is refined to a limited range of k (target: titleType).

3.1.1 *Target:* titleType

We fitted and tested the classifier with the following configuration of parameters:

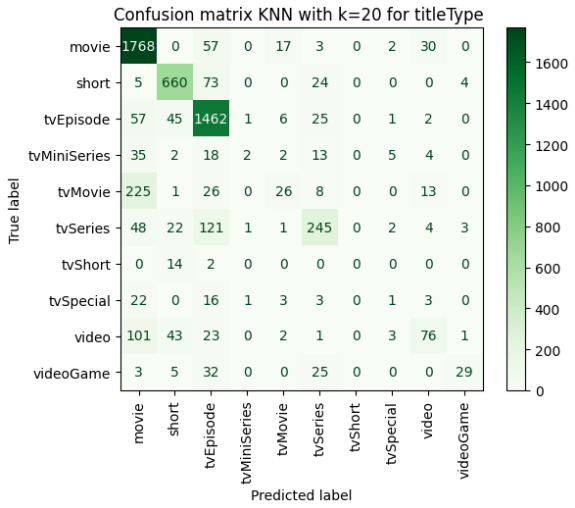


Figure 28: Confusion matrix for KNN for *titleType*

- distance function: Manhattan
- weights: distance
- k: 20 (as can be seen in Fig. 27 k=14 obtained the highest mean accuracy score, but we preferred 20 because of the lower standard deviation).

This model obtained an **accuracy** of **0.78** and a **macro F1 score** of **0.42** on the test set, well exceeding the performance of the dummy classifier.

In particular, as can be seen from the confusion matrix in figure 28, the model has little trouble identifying the three classes with higher support (movies, shorts and TV episodes have all F1 scores higher than 0.84) and is able to correctly detect tvSeries around half of the time. We can also see that TV movies, miniseries and videos are most frequently misclassified as movies, while TV shorts are almost always classified as shorts. TV specials are sometimes misclassified as TV episodes (especially if shorter) and sometimes as movies (especially if longer).

3.1.2 *Target:* binned rating

We fitted and tested the classifier with the following configuration of parameters:

- distance function Manhattan
- weights: distance
- k: 95

This model obtained an **accuracy** of **0.44** and a **macro F1 score** of **0.39**, exceeding the performance of the baseline in both metrics. Overall the performance on this target variable was less satisfactory, as expected.

The inspection of the confusion matrix revealed confusion among all classes (not only adjacent classes), with only *low* and *medium high* obtaining a F1 score greater than .50. The worst performing class was *high*, with an especially bad recall (0.14): more than half of the time titles with a high score were misclassified as *medium high*, with the rest of the cases distributed seemingly at random in the remaining classes.

Model	Accuracy	Macro F1	PR AUC
GNB	0.68	0.40	0.44
CNB	0.77	0.56	0.58

Table 9: Performance of GaussianNB and CategoricalNB models on *titleType*.

3.2 Naïve Bayes Classifier

For each target variable, we built two models: a GaussianNB model using only numerical features (GNB), and a CategoricalNB model using all features except the target variable (CNB). The latter required preprocessing: we discretized the numerical features into quartile-based bins and one-hot encoded categorical features. For *countriesOfOrigin* we only kept the 40 most frequent countries to reduce dimensionality. Given their nature, NB Classifiers don't require feature scaling or feature selection: they rely on probabilities rather than distances and are robust to irrelevant features.

3.2.1 *Target:* titleType

Both models outperform the dummy classifier, but CNB performs significantly better: in fact, GNB has a significantly lower accuracy than KNN, whereas CNB has a comparable accuracy and a significantly higher macro F1 (cf. Table 9).

Analyzing the **GaussianNB** model, we can immediately see that, similarly as for KNN, the classes with a higher support yield better performances (F1 > 0.74, with movie achieving >0.80 for all metrics). The 3rd most popular class, short, performs almost as well as movie while having less than half its support (P: 0.79, R: 0.78): as we've seen in clustering, this class separates itself from the others particularly well. As for the remaining underrepresented classes, they get often misclassified as more popular classes with which they share some similarities: tvMovie and tvShort are almost always classified as movie and short, respectively; tvSpecial, tvSeries and tvMiniS-

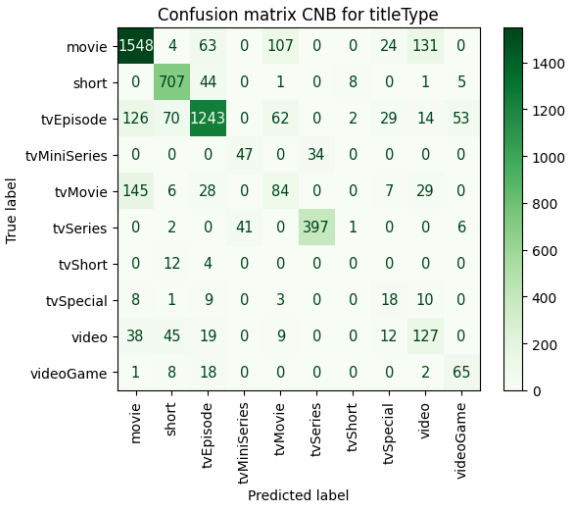


Figure 29: Confusion matrix for CNB for *titleType*.

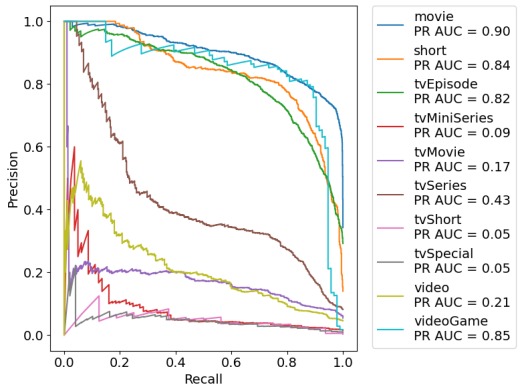


Figure 30: PR curves (1-vs-rest) for GaussianNB.

eries have largely been classified as tvEpisode; videogame has a good precision (0.86) but was also mistaken for tvEpisode a significant number of times.

Moving onto the **CategoricalNB** model, the performance for almost all classes improves, with all three most popular classes now scoring >0.80 in all metrics. The biggest advantage w.r.t KNN is the improvement in performances for lower-support classes: tvMiniSeries and tvSeries are now correctly classified most of the time, and the model confuses them almost exclusively for each other (cf. the confusion matrix in Fig. 29). The latter scores the highest precision (0.921) and F1 (0.904). tvSpecial also improves (F1: 0.26, vs KNN: 0.03 and GNB: 0.11), though there remains significant confusion w.r.t. other classes with higher support.

The two PR curves in Fig. 30 and 31 show this improvement, as well as the drop in performance for videogame w.r.t GNB: it exhibits a noticeably poorer balance between precision and recall.

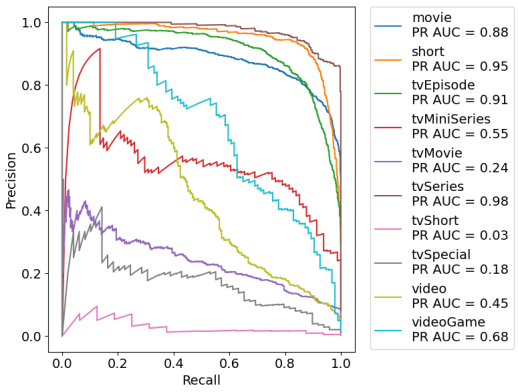


Figure 31: PR curves (1-vs-rest) for CategoricalNB.

Model	Accuracy	Macro F1	PR AUC
GNB	0.35	0.33	0.35
CNB	0.43	0.39	0.40

Table 10: Performance of GaussianNB and CategoricalNB models on *binned rating*.

3.2.2 *Target:* binned rating

As expected, the performance of the two models on this task was less satisfactory, especially for GNB (cf. Table 10).

The **Categorical NB** model yielded very similar results to KNN, with the exception that Medium-Low was the lowest performing class (Recall: 0.15, F1: 0.22): it was correctly classified only 15%, while it was classified as Low and Medium-High 44% and 29% of the times, respectively.

For the **GaussianNB** model, there's confusion among all classes, in particular w.r.t Medium-High, the most popular and best performing class (F1: 0.43). In fact, instances from all classes are predominantly classified as Medium-High. The only exception is the Low class, which still shows a substantial rate of misclassification: 29% of its records are misclassified as Medium-High, vs 33.5% of correct classifications.

The difference in the two NB models' scoring and ranking capability is better visualized by the PR curves in Fig. 32: for CNB, the precision for Low and Medium-High is more stable across different thresholds.

3.3 Decision Tree Classifier

For each task, we trained one model using all features except the target variable, preprocessed as detailed in Subsec. 3.2. Decision Trees (DTs) do not require feature scaling, but they are prone to overfitting, therefore we applied pre-pruning and post-pruning techniques. For each model, we

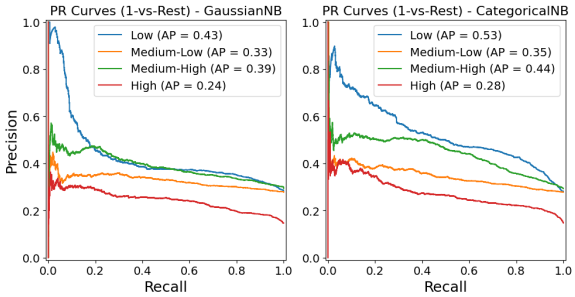


Figure 32: PR curves for GaussianNB and CategoricalNB for *binned rating*.

used a random search to find the best combination of hyperparameters (i.e. the one yielding the best accuracy) from the following grid:

```
{'max_depth': [None] + list(np.arange(2, 20)),
'min_samples_split': [2, 5, 10, 15, 20, 30, 50, 100],
'min_samples_leaf': [1, 5, 10, 15, 20, 30, 50, 100],
'criterion': ['gini', 'entropy']}
```

The random search was performed with 5-fold stratified cross-validation, repeated 5 times. Keeping the best hyperparameters found fixed, we then searched for the value of the *ccp_alpha* parameter that maximized the average macro F1 score of the validation sets in a 5-fold stratified cross-validation. Since we'd already implemented pre-pruning, we favored the macro F1 score rather than the accuracy because it lead to a more conservative post-pruning. The resulting hyperparameter configurations are reported in Table 11.

Model	max_ depth	min_s _split	min_s _leaf	criterion	ccp_ alpha
titleType	13	30	5	entropy	3.37e-4
bin_rating	11	100	5	entropy	4.49e-4

Table 11: Best hyperparameters for DTs.

3.3.1 *Target:* titleType

The DT outperformed all previous models, both in terms of accuracy and macro F1, as well as in terms of PR-AUC (cf. Table 12). Similarly to the other models, the DT does not struggle with the three most popular classes (F1 > 0.85), but moreover shows a significant improvement for the lower support classes (cf. the confusion matrix in Fig. 33). In fact, the best performing class is videogame (F1 = 0.95, vs KNN = 0.44 and CNB = 0.58), which is classified correctly most of the time. Following the precedent set by the CNB, it also has learned to distinguish tvMiniSeries and tvSeries from the rest, with the latter having the

Model	Accuracy	Macro F1	PR AUC
titleType	0.85	0.66	0.68
bin_rating	0.44	0.41	0.43

Table 12: Performance of DTs on *titleType* and *binned rating*.

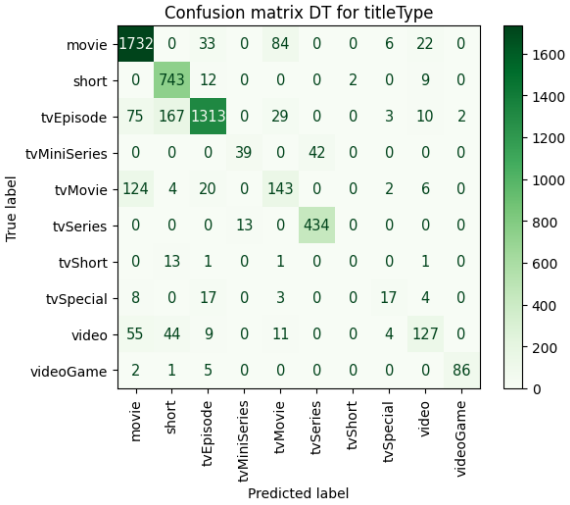


Figure 33: Confusion matrix for DT for *titleType*.

2nd highest macro F1 (0.94): they now both only get misclassified as each other. tvSpecial (F1 = 0.42, vs KNN=0.03 and CNB = 0.26) is mostly confused between itself and tvEpisode.

Unsurprisingly, the most important features are *logRuntime* (0.45), *canHaveEpisodes* (0.18) and the genre *Short* (0.17). The first split on *logRuntime* $\leq$ 4.119 ($\approx$ 60.5 minutes) already allows to separate all shorts, tvShort, and videogames in the left subtree. In two more splits, we get to a node in the right subtree which will lead to a leaf: it's adult videos that are longer than an hour and released since 1991. On the left subtree, we get to a leaf containing *tvSeries* in three total splits: they are at most an hour long, have *Short* as a genre, and can have episodes. It's also worth noting that all (except one) videogames are at most an hour long and are not tagged as genre *Short*.

3.3.2 *Target:* binned rating

The model's performance is comparable to that of the KNN and CNB models (cf. Table 12). W.r.t them, it achieves a slightly better F1 score for Medium-Low (F1: 0.34, vs KNN: 0.3 and CNB: 0.22) thanks to its higher recall, more balanced with the precision.

The three most important features are *tvEpisode* (0.22), *startYear* (0.14) and *numVotes* (0.14). Looking at the first two levels of the DT (Fig. 35), it

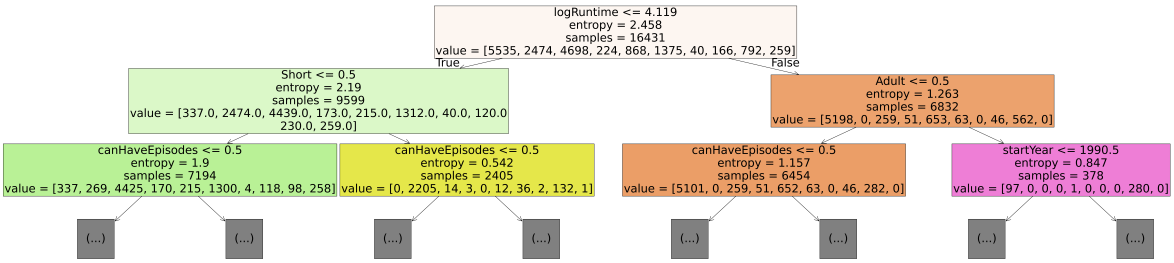


Figure 34: DT classifier for *titleType*, cut at depth 2 (left= True, right= False).

is evident that the nodes are not very pure. The first split isolates the most part of the Low and Medium-Low classes in the non-tvEpisode subtree (91% and 77.6%, respectively): this likely explains why Low is the best performing class. The subsequent split on movie further separates the High class: around 43% of all High rated records are neither tvEpisodes, nor movies.

3.4 Conclusions

As anticipated, the task of predicting *titleType* yielded better results across all models, thanks to clearer class separability and stronger feature associations. Nevertheless, some classes had too limited of a support to allow for generalization (e.g. tvShort: all models scored 0% precision and recall). The Decision Tree Classifier achieved the best results, especially in terms of macro F1. It is followed by the CNB, with KNN trailing by a small margin. The differences in performance among the three models were less accentuated for the *binned rating* task, while the GNB resulted in consistently lower performances.

4 REGRESSION

We performed two univariate regression tasks employing both linear (ordinary least squares and Ridge regression) and non linear methods (K-Nearest Neighbor and Decision Tree). In the first task we attempted to predict a title's **average rating** and in the second the (log-transformed) **number of votes** received by a title.

As evaluation metrics for our models, we used the Mean Squared Error (MSE) and the Determination Coefficient (R2). As an absolute minimum baseline we would like for the MSE to be lower than the target variable variance (1.84 for average rating, 3.1 for numVotes)

4.1 K-Nearest Neighbor Regressor

For each target feature we trained the KNN regressor on all numerical features (excluding the target and log transformed if necessary) which were normalized using a StandardScaler. In both cases we performed grid search in order to find the optimal configuration of hyperparameters as described in Section 3.1, except that in this case we evaluated the candidate models on the basis of negative mean squared error.

Since the regressor for **average rating** showed a very bad performance and no sensibility to the configuration of hyperparameters, and the performance increased monotonically with k, which meant that the best model we could obtain was a costly way of calculating the mean of the target variable, we fitted and tested a KNN regressor using the library defaults.

This model obtained a **R2 score** of **-0.2** and a **MSE** of **2.2**, which means that the predictions of the models are worse than predicting always the mean value of the target variable (since the MSE is higher than the variance of the target variable). As can be seen in Fig. 36, there is no correlation between real average rating values and those predicted by the KNN regressor. In this figure we can also see that the predicted values are squeezed towards the mean rating of the entire dataset due to the strategy used by the regressor for the prediction (mean score among the 5 nearest neighbors).

For *numVotes* we fitted and tested a regressor with the following configuration of parameters:

- distance function: Manhattan
- weights: distance
- k: 19

The performance on this task was much better: the model obtained a **R2 score** of **0.69** and a **MSE** of **0.96**. This was an expected result, as we saw in Section 1.5 a number of features has moderate to high positive correlation with our target

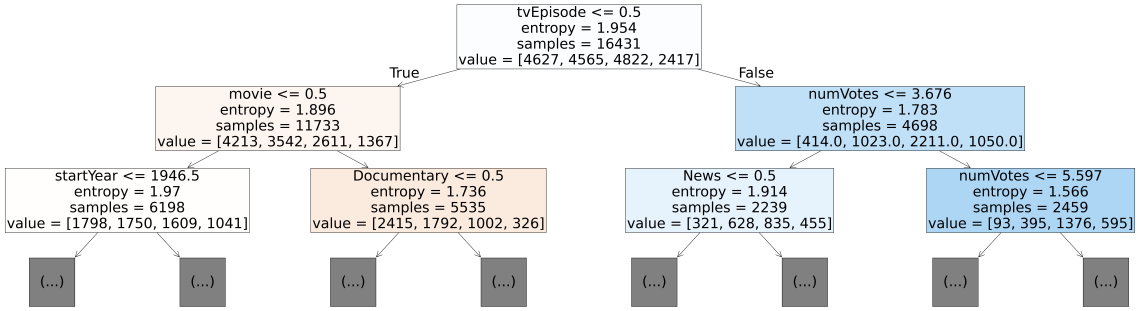


Figure 35: DT classifier for *binned rating*, cut at depth 2 (left= True, right= False).

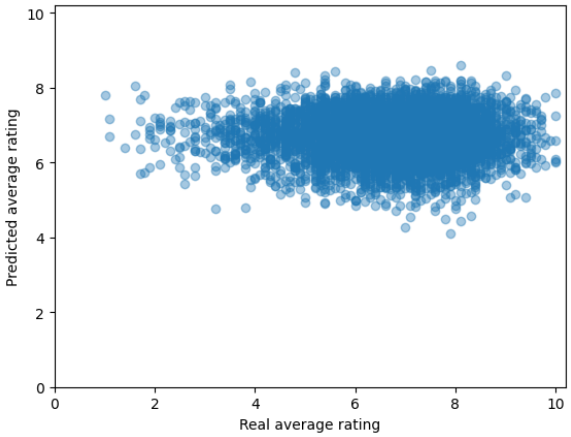
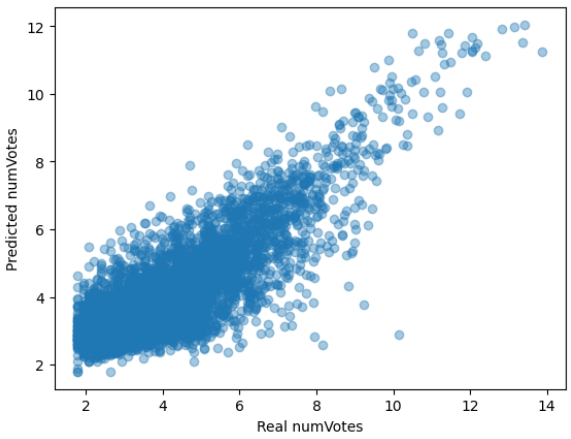


Figure 36: Relationship between real and predicted values of average rating (KNN regressor).



feature. As can be seen from Fig. 36, real and predicted values are highly and positively correlated (Pearson R: .83).

4.2 Decision Tree Regressor

We trained the DT regressor on all (categorical and numerical) features with the same preprocessing described for classification. In order to determine the best combination of parameters, we performed random search on the same hyperparameter space defined for the classification task (except we used only MSE as splitting criterion).

For the **average rating**, we fitted and tested a DT regressor with the following parameters:

- max_depth: 2
- min_samples_split: 2
- min_samples_leaf: 100

Again, the model performace proved unsatisfactory, with a **R2 score** of **0** and a **MSE** of **1.83**.

For the **number of votes**, we fitted and tested a DT regressor with the following parameters:

- max_depth: 11
- min_samples_split: 50
- min_samples_leaf: 15

which performed slightly worse than the KNN regressor: **R2: .67**, **MSE: 1**. We inspected the features with higher feature importance (calculated as total reduction in negative MSE brought by each feature) in order to determine which numerical features we should attempt to use for the linear regression. According to the model the three top features were *criticReviewsRatio*, *numRegions* and *totalCredits*.

Feature combination	OLS	Ridge
top DT	0.459	0.459 ($\alpha = 10$)
top correlated	0.518	0.519 ($\alpha = 10$)
all	0.547	0.548 ($\alpha = 10$)

Table 13: Validation R2 scores for all linear regressors.

4.3 Linear models

Since none of our numerical features showed any correlation with the mean rating, we attempted to fit an ordinary least squares linear regressor using all numeric variables as input. This model obtained a **R2 score** of **0** and a **MSE** of **1.83**.

For **numVotes**, we performed model selection among models using different combinations of input variables and regularization (since some of our input variables are correlated with each other, we used Ridge regularization).

We used the following combinations of features:

1. Top 3 DT features: *criticReviewsRatio*, *numRegions*, *totalCredits*;
2. Top 3 correlated features: *totalImages*, *numRegions*, *totalCredits*;
3. all numerical features.

For each combination of input variables we performed 5-fold cross validation on the training set with and ordinary least squares linear regressor and reported the R2 score.

For the Ridge regressor, we used the leave-one-out cross-validation built in sklearn `RidgeCV` in order to determine the best value of α (checking for powers of 10 between 10^{-3} and 100), in all cases we found that the best α was 10.

Table 13 reports the scores obtained in validation: we can see that applying regularization had little effect on the performance of our models and that the best performing model uses all numerical features in input and Ridge regularization with α=10.

We then tested the best model, obtaining a **R2 score** of **0.55** and a **MSE** of **1.389**.

4.3.1 Conclusions

We were not able to fit any regressor that could predict the average rating of a title adequately.

On the other hand, when using *numVotes* as target variable we were able to obtain good results. As evidenced by the reported scores, non-linear models performed much better than linear ones.

KNN performed slightly better than DT. Considering that this slight improvement comes at the cost of less interpretable results and greater computational cost at prediction time, DT is still the better model.

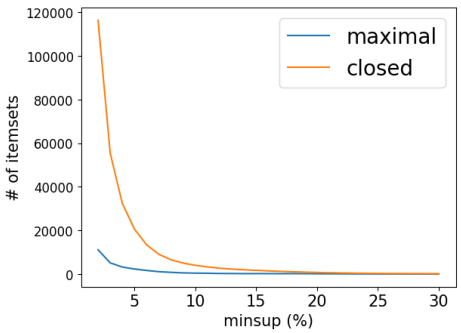


Figure 37: N. of closed and maximal frequent itemsets for different *minsup* values.

5 PATTERN MINING

In this section, we will detail our experiment using the **FP-Growth** algorithm to extract frequent itemsets and association rules. The most useful rules were then exploited to classify some relevant *titleTypes*.

Preprocessing was required to turn the dataset into a list of transactions, with each record representing a transaction containing one item for each feature value associated with that record. We have two multilabel variables, *genres* and *countryOfOrigin*, and we decided to only keep the four most frequent countries to reduce dimensionality, aggregating the rest into an 'Other' category. Numeric variables were discretized into quartile-based bins, and, to allow interpretability, the name of the variables was appended to their values.

5.1 Frequent Itemsets Extraction

We kept the minimum number of items in the itemsets (*zmin*) fixed to two, as 2-itemsets could already be interesting. Fig. 37 shows the number of closed and maximal frequent itemsets extracted for different *minsup* values, ranging from 2% to 30%. The number of itemsets drops quickly, and after 25% the two groups seem to start converging to lower values (< 250). We also looked at the maximal frequent itemsets containing our target variable, *titleType* (Fig. 38): there are none for the minority classes tvShort, videoGame, tvMiniSeries, tvSpecial, and we start losing more classes from *minsup* > 4%.

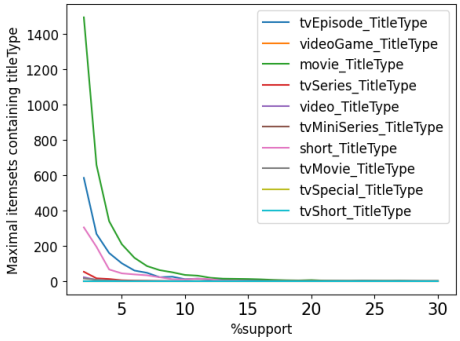


Figure 38: N. of maximal frequent itemsets containing *titleType*for different *minsup* values.

Rank	Maximal Itemset	Sup
1	(movie_TitleType, NoEpisodes)	33.686
4	(country_Other, NoVideos, OneCountryofOrigin)	32.481
6	(genre_Drama, NoEpisodes, OneCountryofOrigin)	31.514
7	((50.0, 90.0]_RuntimeMin, NoEpisodes)	31.471

Table 14: Selected top maximal itemsets for *minsup*=30.

After testing different *minsup* values, we chose 30%, as we got some interesting maximal itemsets, rather than just a combination of the most frequent values for the different features. At this threshold, the frequent itemsets are 174 and they are all closed, while the maximal itemsets are only 26. In Table 14 are reported some of the most interesting maximal itemsets, among the top 10 with the highest support. The first one captures a dependency we expect: a movie doesn't have episodes. This is similarly captured by the 7th, as the reported runtime is typical of movies and tv episodes. The 4th one represents many foreign titles, made in just one foreign country, while the 6th captures dramatic stand-alone titles from a single country: it can well describe telenovela episodes, or lower budget dramatic movies (as they are made in a single country).

5.2 Association Rule Extraction

The heatmap in Fig. 39 shows the number of association rules extracted by the FP-Growth algorithm for different *minsup* and confidence (*minconf*) thresholds. It is evident that the number of rules drops quickly as the *minsup* increases, while it is less affected by *minconf*. Therefore, we first kept a fixed low value of confidence (65%) and experimented with different values of *minsup*, increasing it by small increments. Once we found

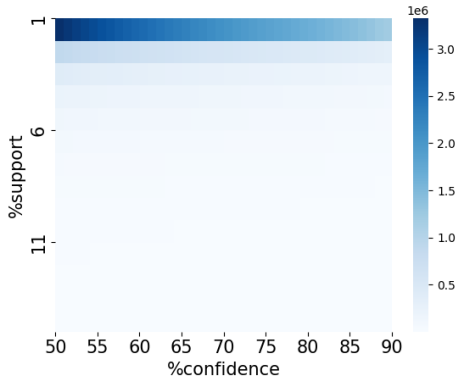


Figure 39: N. of association rules extracted for different *minsup* and *minconf* values.

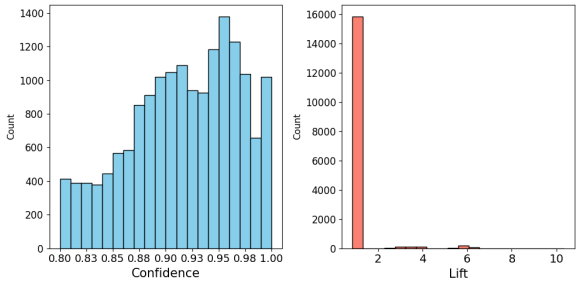


Figure 40: Histograms of rules' confidence and lift.

a value yielding an interesting list of rules, we increased the confidence threshold. We reached 80% *minconf* before losing interesting rules. The final hyperparameters are: *zmin*=2, *minsup*=9% and *minconf*=80%.

The total number of rules extracted is 16466. The histograms in Fig. 40 show the rules' confidence and lift distributions. While most rules exhibit a confidence greater than 0.87, the vast majority are weak, with a lift below 1.

The 30 rules with the highest lift all contain a *titleType* either in the antecedent or consequent; a selection is reported in Table 15. In particular, the two top rules have captured the fact that tvSeries have episodes, while the rest have captured the redundancy of the information about shorts, present in both *genres* and *titleType*. In particular, rules having *short_titleType* as consequent are all associated with both *genre_Short* and the lowest value for *runtimeMinutes*, [0, 29].

5.3 Classification: *titleType*

Out of the extracted rules, 245 have a *titleType* in the consequent: 144 have short, 89 have tvEpisode, 10 have movie, and two have tvSeries. For each of these four *titleType* classes, we implemented the rule with the highest support for a task of

Antecedent	Conseq.	%sup	conf	lift
(HasEpisodes, OneCountryofOrigin)	tvSeries _TitleType	7.89	0.866	10.35
(HasEpisodes)	tvSeries _TitleType	8.37	0.860	10.28
(short_TitleType, [0, 0.693]_Total-Images, NoEpisodes)	genre _Short	8.48	0.931	6.21
(genre_Short, [0, 29]_RuntimeMin, (0.692, 1.386]_Num-Regions, NoEpisodes, OneCountryofOrigin)	short _TitleType	11.33	0.933	6.20
(short_TitleType, [0, 2.833]_TotalCred-its, NoEpisodes)	genre _Short	8.58	0.928	6.19

Table 15: Selected top association rules by lift.

Antecedent	Conseq.	%sup	conf	lift
(genre_Short, NoEpisodes)	short _TitleType	13.42	0.900	5.980
((29.0, 50.0]_ RuntimeMin, NoEpisodes)	tvEpisode _TitleType	16.57	0.873	3.054
((90.0, 3000.0]_ RuntimeMin, NoEpisodes)	movie _TitleType	12.85	0.824	2.446
(HasEpisodes)	tvSeries _TitleType	8.37	0.860	10.276

Table 16: Association rules for *titleType* classification.

multiclass classification, for the sake of which the remaining classes were aggregated to 'Other'. We favored the rules with the highest support because they are more general and include the most important items in the antecedent; nevertheless, we made sure that their lift was higher than 1. The selected rules are reported in Table 16.

The performance of our simple rule-based classifier on the test set is reported in Table 17. It achieved an accuracy of 60% and a macro F1 score of 67.5%, outperforming our baseline model, a stratified dummy classifier trained on the training set, across all metrics for all classes (accuracy: 0.25, macro F1: 0.20).

	precision	recall	f1-score	support
movie	0.808	0.397	0.532	1877
other	0.210	0.607	0.312	789
short	0.890	0.911	0.901	766
tvEpisode	0.952	0.573	0.715	1599
tvSeries	0.847	1.000	0.917	447
accuracy			**0.600**	5478
macro avg	0.741	0.698	**0.675**	5478

Table 17: Classification report for rule-based classification.

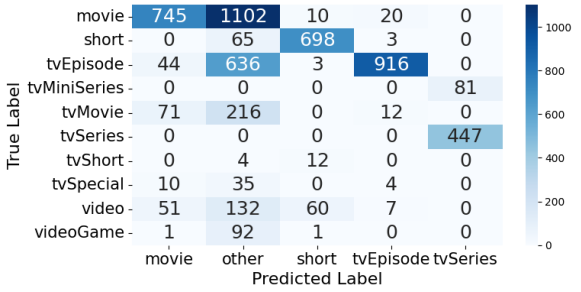


Figure 41: Confusion Matrix for rule-based classifier (10 True × 5 Predicted).

The confusion matrix in Fig. 41 allows us to understand what types of mistakes the classifier made and how the 'Other' classes were classified. The information about having episodes was enough to achieve perfect recall for tvSeries, but of course all tvMiniSeries were also classified as such. The same happens for tvShorts, all classified as shorts – the same had happened for our other classifiers. The information about *runtimeMinutes* has lead to some misclassification, as the binning is quite coarse-grained; this explains the lower recall for tvEpisodes and, especially, for movies: there's plenty of movies with a runtime < 90 minutes. All of our classification models have yielded better performances for these two classes: the information carried by the other features proved to be useful.

In conclusion, the selected association rules leveraged on runtime, the information about serialized format, and the explicit information about shorts: these were exactly the top three most important features for our DT classifier. Using significantly less information than our classifiers, they were able to yield satisfactory results for these relevant classes.